# From Detecting Performance Regressions to Explaining Optimizations: An Empirical Study with PerfLens

Ansar Bashayev, Thanh Tra Nguyen, and Dasol Jung

Hochschule für Angewandte Wissenschaften Kiel (HAW Kiel),
Sokratesplatz 1, 24149 Kiel, Germany
Advanced Software Engineering Seminar

**Abstract.** We extend a NetworkX regression-detection prototype (paired repetitions, bootstrap interval, ±5% threshold) into *PerfLens*, which adds Java, correctness checks, environment records and separate profiling. On two machines, a controlled suite detected known effects, but unchanged-code controls were inconclusive on the noisier one. CPU contention roughly doubled measured time; moving the load reduced mean time without an established reduction of variability. In RoaringBitmap pull request #845, the optimized revision reduced the time of `rank(65535)` by 99.49% under Docker/WSL2 (95% CI −99.50 to −99.48%), a near-boundary control stayed within threshold, and removing the new guard restored the cost. For the first call on an invalid cardinality cache, however, the change was −16.62% with an interval crossing zero, while the next call was 98.73% faster. Native Windows on the same computer reproduced both large effects, but its control was practically inconclusive. Reliable claims thus need a stated workload and state, independent repetition, controls and a tested mechanism.

**Keywords:** Performance regression detection · Benchmarking · Measurement noise · JMH · Reproducibility

## 1 Introduction

Software performance changes with every revision. A newer version may become slower, faster, or only *appear* different because hardware, scheduling, caches, just-in-time (JIT) compilation and garbage collection make measurements vary [1,2]. A single timing comparison cannot decide whether a change is real.

Our project began as *An Empirical Study of Performance Regressions in NetworkX* [15]. The preliminary report asked how a difference between two versions can be identified as a real regression rather than measurement variation, selected NetworkX, and planned a lightweight Python framework for comparing Git revisions; a first prototype then compared NetworkX and NumPy versions [16]. After seminar feedback and alignment with the official Topic 9 question—*How can software performance be measured reliably, and which optimization techniques provide reproducible improvements?* [14]—we widened the scope: (i)

sources of measurement noise and their mitigation; (ii) explaining a change, not only detecting it; (iii) evaluating optimizations without a prior regression; and (iv) Java support and a real open-source optimization under different runtimes and environments. For Java we selected RoaringBitmap pull request #845 [13].

We refine the official question into three research questions that absorb the preliminary report's Q1–Q4 [15]:

- **RQ1** (measurement and classification; prior Q1, Q3, Q4): How can paired repeated benchmarking with an uncertainty estimate separate meaningful changes between two versions from measurement variability in Python and Java?
- **RQ2** (variability; prior Q2 and part of Q1): How large is the variability, and how do interference, warmup, runtime configuration and environment change measured performance and its interpretation?
- **RQ3** (reproducible optimization): Can a real library optimization be reproduced and its source-level mechanism isolated, and in which states and environments does its benefit hold?

We contribute an auditable workflow that extends the prototype without replacing it, a controlled evaluation of detection and noise, and a mechanism-level study of a real Java optimization, including negative and uncertain results. We do *not* claim an accurate general detector, production speedups, or replication of the Java case on other machines.

## 2 Background and Related Work

### 2.1 Benchmarks, variability and significance

A benchmark combines a workload, the software under test and a measurement procedure; its measurements contain noise from CPU frequency, caches, scheduling, background activity, JIT compilation, garbage collection and the benchmark configuration (iterations, warmup, input) [15]. For reference and candidate mean times $\bar{t}_A$ and $\bar{t}_B$ we report the relative change

$$\Delta_t = 100\left(\bar{t}_B/\bar{t}_A - 1\right)\%, \tag{1}$$

where a negative value means less time. A decision must ask whether a difference is *statistically* reliable and whether it is *practically* relevant; with many samples a 1% change can be clear yet unimportant.

### 2.2 Rigorous measurement

Georges, Buytaert and Eeckhout [1] show that inconsistent Java evaluation practice and nondeterministic runtime behavior can change conclusions; they separate startup from steady state and recommend repeated VM invocations summarized with confidence intervals rather than, e.g., the best run. Kalibera and Jones [2] model variation at several levels (build, VM execution, iteration),

allocate repetitions where they reduce uncertainty most cheaply, and report effect sizes with intervals. Both require independent repetition and uncertainty, but Georges et al. focus on *what* phase is measured and how to summarize it, Kalibera and Jones on *how much* repetition is needed at which level and at what cost. Neither justifies treating iterations inside one JVM as independent samples. Repetition has limits: seemingly irrelevant setup choices bias measurements, so a narrow interval does not exclude systematic error [3], and VM warmup often reaches no steady state after a fixed number of iterations [4]. We adopt the shared principles (fresh processes as units, paired intervals, recorded warmup) but not Kalibera and Jones' full variance allocation, and we cannot prove steady state.

### 2.3 Detecting and explaining regressions

Chen and Shang [5] found in real project histories that regressions often come from ordinary maintenance such as bug fixes. Perphecy [6] and PerfJIT [7] reduce detection cost by predicting which commits or tests are likely to affect performance. Laaber and Leitner [8] show that microbenchmark suites differ in stability and in detecting injected slowdowns. NetworkX maintains an Airspeed Velocity (ASV) suite tracking performance across commits [10]. The systematic mapping of dos Santos et al. [9] identifies open problems in test granularity, testing frequency, detection accuracy and cost. Prediction and test selection still rely on a trustworthy measurement, the gap our preliminary report addressed. Detection is also not explanation: Heger et al. [11] isolate root causes by combining unit tests with revision history, and Coz [12] estimates the impact of code regions through controlled perturbation. A timing difference, a suspicious revision, a profiler hotspot and an intervention are different levels of evidence; for a small change, we use an intervention that removes only the suspected optimization.

## 3 From the NetworkX Prototype to PerfLens

### 3.1 Preliminary study and first prototype

The team's project plan [18] defined ten repository criteria: C1 Git history, C2 performance-related history, C3 benchmark availability, C4 cross-version compatibility, C5 environment reproducibility, C6 execution cost, C7 benchmark diversity, C8 commit-level traceability, C9 activity and C10 practical feasibility. NetworkX, SymPy and jsonschema were compared on them [15]. In a suitability pilot, the same workload ran unmodified on two releases of each (NetworkX 3.5/3.6, SymPy 1.13.3/1.14.0, jsonschema 4.25.1/4.26.0) with 20 repeated measurements each. All three passed, so C4 and C6 did not separate them. NetworkX was preferred for its lightweight environment, diverse benchmarks and ASV support. The pilot assessed suitability, not performance; its raw data are not in the archive.

The prototype [16] implemented this workflow. It timed a shortest-path calculation on a grid graph with warmup and 20 paired rounds, estimated a 95% paired bootstrap interval from 5,000 resamples, and classified results with a

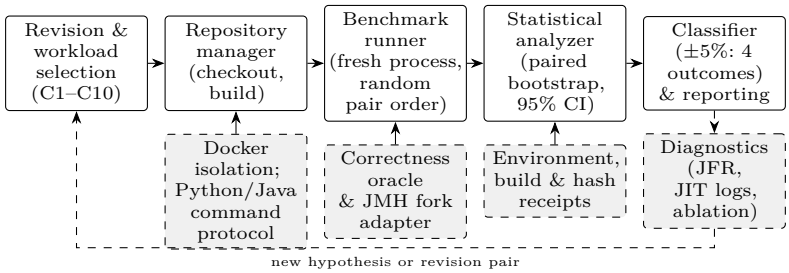



**Fig. 1.** PerfLens structure. Solid boxes: components of the original prototype (kept, analyzer code unchanged). Dashed boxes: extensions. A classification triggers diagnosis and possibly a new optimization hypothesis; diagnostic runs are kept apart from clean timing.

configurable ±5% threshold. Three cases were reported (values from the presentation [16] and handoff record [17]; raw observations not in the examined package, hence not recomputed): NetworkX changed by +0.56% $[-2.71, +4.25]$% (*within threshold*); a NumPy `argsort` release comparison showed +236.08% $[+225.44, +247.75]$% (*regression*); and a NumPy unchanged-version comparison gave +9.93% $[-1.77, +26.22]$%, flagged by its point estimate but *inconclusive* by its interval. Stated limitations were few cases, fixed 20 rounds and threshold, environmental noise, and no identified regression-introducing commit. The planned characterization of regression frequency and distribution across NetworkX history was *not* carried out; the NetworkX work validated the workflow and shaped the framework. Per the handoff record, reruns in a new environment repeated these decisions (NetworkX −0.64 and −1.19%, within threshold; NumPy +232.03%).

### 3.2 The extended PerfLens workflow

Figure 1 shows the current structure. The original components remain, including the unchanged statistical analyzer (the new runner imports its `compare` function) and the four-way classifier. PerfLens adds a Python/Java command runner, correctness oracles that must pass before timing counts, environment and build receipts with hashes, a JMH adapter treating one JVM fork as one observation, failure records, and a diagnostic path (JFR, compilation logs, profiles) kept apart from clean timing. Third-party Java code runs in a network-disabled Docker container with a 2-CPU quota.

A percentile bootstrap resamples whole pairs (one fresh observation per arm, random order) 5,000 times. Interval and predeclared threshold give four outcomes: *regression* (interval above +5%), *improvement* (below −5%), *within threshold* (inside ±5%) and *inconclusive* (otherwise). An interval that excludes zero but crosses a ±5% boundary shows a statistical direction without a practical decision. Table 1 lists the evidence; machines and environments are never pooled.

**Table 1.** Evidence inventory. Observations are fresh processes or JMH forks, not inner iterations.

| Study | Environment | Comparisons / obs. |
|---|---|---|
| Prototype (NetworkX, NumPy) | original team machines | 3 cases |
| Controlled suite, 2 sessions, 20 pairs; run per host | home PC (AMD Ryzen 5 5500); office PC (Intel Core Ultra 7 155H) | 17 / 680 each |
| Native Python/Java studies (noise, runtime, detector) | office PC | 22 / 278 |
| Synthetic detector calibration | generated data, no hardware | 540 datasets |
| RoaringBitmap #845, JMH 1.37 | office PC, Docker/WSL2 | 9 / 156 forks |
| Cache states; native replication | office PC, Docker; Windows 11 | 3 / 60 forks each |

## 4 Experimental Design

### 4.1 Controlled suite and detector evaluation

The suite was fixed before execution: nine comparison types, two arms, 20 paired rounds of fresh processes; eight were repeated in a second session with a new order seed. Python cases use a NetworkX breadth-first search (BFS) on a $35 \times 35$ grid: an unchanged-code control, an injected second BFS per query (known slowdown), and a correctness-checked reuse of one BFS for 16 targets (plus a $60 \times 60$ variant). Java cases compare linear with binary search on sorted data, 0 with 20 warmup calls, and `-Xint` with the default JVM. CPU interventions pin the workload to a busy loop's logical CPU (contention) or to another CPU (mitigation). The suite ran on a home PC and, unchanged, on an office PC; a further office rerun was the baseline for later work. Native office studies added 15 small extra-work interventions (five pairs each), runtime-setting selection with held-out confirmation, and CPU-class placement. A synthetic study generated 540 datasets (18 cells $\times$ 30; true effects $-15$ to $+30\%$; CV 1, 5, 20%) to observe the unchanged classifier under a known model.

### 4.2 Real Java case: RoaringBitmap PR #845

Java candidates were screened with C1–C10 and the plan's keyword search [18]: RoaringBitmap (PRs 845, 840), Caffeine (issue 1320) and Apache Fory (PR 3717). PR #845 was the most bounded case: an intended performance change, an exact parent/child commit pair (before `2dfd0fb99446`, after `684531c13877`), unchanged functional results, and one harness for both revisions. In the old code, `BitmapContainer.rank(65535)` counts the bits of all 1,024 64-bit words; the new code returns the stored cardinality when it is valid. The query `rank(65534)` keeps the scan in both revisions and acts as a negative control, and a full-container `rangeCardinality` call exercises the shortcut through a public API. Both revisions were built in Docker and passed an independent membership oracle and the upstream rank tests (193 before, 195 after); four nearby revisions passed the same harness pilot. The measured artifact is a `javac -release 8`

**Table 2.** Controlled suite, session 1: relative time change [95% CI] and decision (R regression, I improvement, W within threshold, ? inconclusive). Each cell: 20 pairs.

| Comparison (reference → candidate) | Home PC | Office PC |
|---|---|---|
| Unchanged NetworkX BFS | −0.08 [−1.14, +1.05] W | −7.60 [−21.84, +15.36] ? |
| Inject second BFS per query | +101.99 [+94.12, +112.05] R | +121.47 [+88.85, +157.11] R |
| Reuse one BFS for 16 targets | −93.63 [−94.00, −93.06] I | −94.32 [−94.99, −93.60] I |
| Java linear → binary search | −97.60 [−97.63, −97.56] I | −97.10 [−97.44, −96.70] I |
| Quiet → load on same CPU | +98.00 [+85.11, +114.06] R | +52.30 [+20.90, +87.00] R |
| Loaded CPU → other CPU | −50.84 [−55.17, −45.72] I | −38.13 [−44.70, −30.38] I |
| Java 0 → 20 warmup calls | −38.79 [−41.43, −36.23] I | −49.85 [−56.26, −42.26] I |
| Java `-Xint` → default JVM | −82.38 [−82.72, −82.00] I | −75.84 [−84.25, −65.82] I |

build of the base sources on Temurin 21.0.7, not a published JAR. JMH used one fork per observation, three 200 ms warmup and three 300 ms measurement iterations, a 512 MiB heap and one thread. Further inputs were 1,428,579 bytes of the project's Apache-2.0 Java sources (identical in both arms; checked by 44 boundary cases) and a guard-ablation variant of the new revision without the `rank(65535)` shortcut. A separate dataset collection was not used because its license was unclear.

### 4.3 Cache state and Windows replication

As only the cached case was measured, a final experiment separated three states: a valid precomputed cardinality, a cardinality invalidated to −1 before the first timed query, and the next query after an untimed priming call. A JMH `SingleShotTime` benchmark times one batch of 256 calls per shot on freshly constructed containers (16 reused backing arrays), after 16,000 untimed preheat calls, 30 warmup and 20 measured shots, with ten randomized pairs per state. This measures logical invalidation in a warm JVM, not a cold JVM or memory; an unstable pilot with weaker preheating is excluded. The core workloads were also run natively on Windows 11 (Oracle HotSpot 21.0.8) on the same office PC with the same JMH settings and verified artifacts. A first native attempt, blocked by the sandbox before timing, is kept as a failure record.

## 5 Results

### 5.1 Detection on the controlled suite (RQ1)

Table 2 reports session 1 on both machines. Known effects were classified as expected on both. The unchanged-code controls, however, were within threshold on the home PC (−0.08%, −0.21%) but inconclusive on the office PC (−7.60% [−21.84, +15.36], −0.94% [−14.65, +15.50]), and also in the later office rerun. Decisions agreed for 15 of 17 comparisons, but effect sizes differed (contention +98.00 vs. +52.30%): the decisive difference was the host's noise, not the detector.

"No regression" did not end the investigation: the unchanged BFS workload (about 8.2 ms) missed a 3 ms illustrative budget fixed before timing. Profiling

found 160 BFS calls for ten operations; the reuse variant needed ten, matched an independent distance oracle and took 0.52–0.54 ms in the two sessions—an optimization of our workload, not of NetworkX.

The detector has clear limits. In the synthetic study, a +10% slowdown was flagged in 30/30 datasets at 1% CV, 23/30 at 5% and 7/30 at 20% (23/30 inconclusive); empirical coverage of the nominal 95% interval ranged from 83.3% to 100% across cells. With five measured pairs, a 50% extra-work intervention was detected in 3/3 seeds, 15% in 1/3, and smaller ones in none; one zero-extra-work case even had an interval excluding zero $[-10.56, -1.34]$ that the threshold kept inconclusive. A five-pair identical-code Java smoke test on Linux initially flagged an improvement. Intervals and thresholds prevent many false claims, but do not make small samples accurate.

### 5.2 Sources of variability and mitigation (RQ2)

CPU contention was the largest controlled noise source (Table 2). Moving the workload on the home PC lowered the coefficient of variation (CV) from 20.69 to 6.39% and from 15.42 to 2.45% (sessions 1, 2), but the exploratory CV-ratio interval of session 2 included one: reduced variability is not established. On the office PC, moving contention within the same reported CPU class reduced time by 59.06% $[-62.02, -56.43]$, whereas a policy across heterogeneous core classes gave +12.62% $[+0.82, +35.14]$ (inconclusive). In the RoaringBitmap container, contention on the shared virtual CPU added 87.95% $[+66.76, +113.19]$ and moving the load lowered mean time by 48.61% $[-50.98, -46.03]$; again the CV-ratio interval $[0.693, 1.886]$ includes one. Virtual-CPU placement cannot identify the physical core, frequency or cache effect involved.

Runtime settings matter as much as code. Twenty untimed warmup calls reduced Java time by 35–52% across hosts and sessions, but on the home PC their CV-ratio intervals included one, and fixed warmup does not prove a steady state [4]. A Tier-1-only JIT was rejected and confirmed slower on held-out data (+10.18% $[+6.46, +14.69]$); on the RoaringBitmap workload it increased time by 880.92% $[+853.77, +919.10]$, with compilation logs showing tier 1 instead of tier 4. This supports the default JIT for these workloads, not a general tuning rule.

### 5.3 A real optimization: RoaringBitmap PR #845 (RQ3)

Table 3 (upper block) summarizes the Docker/WSL2 comparisons. For the cached dense container, `rank(65535)` fell from 342.61 to 1.75 ns/op (−99.49%), and the public full-container range query improved by 97.85%. The near-boundary control changed by +0.70%, inside ±5%. Without the guard, the new revision lost the gain: the intact version took 99.44% less time than the ablated one. With the control, bytecode inspection and separate JFR samples, this supports a bounded explanation: the guard and cached field cause most of this operation-level gain. It does not identify each JIT or hardware contributor; JFR sample counts are not speed estimates. With the source-derived input, the full-block query kept a

**Table 3.** RoaringBitmap PR #845 (JMH `AverageTime`, mean ns/op). Core cases 10 pairs, follow-ups 8 pairs. Blocks are estimated separately and never pooled; all ran on the same physical computer. "First" is an invalid cache in a warm JVM, not process startup.

| Workload (reference → candidate) | Ref. | Cand. | Change [95% CI] | Decision |
|---|---|---|---|---|
| `rank(65535)`, before → after | 342.61 | 1.75 | −99.49 [−99.50, −99.48] | improvement |
| `rank(65534)` control | 348.63 | 351.07 | +0.70 [−0.93, +2.58] | within thr. |
| Full-container range | 366.36 | 7.86 | −97.85 [−97.90, −97.81] | improvement |
| Guard removed → intact after | 651.10 | 3.64 | −99.44 [−99.52, −99.35] | improvement |
| Source input, full block | 524.76 | 11.20 | −97.87 [−98.02, −97.68] | improvement |
| Source input, near boundary | 422.70 | 407.75 | −3.54 [−7.18, +0.12] | inconclusive |
| *Native Windows 11, Oracle HotSpot 21.0.8 (Docker/WSL2 above: Temurin 21.0.7)* | | | | |
| `rank(65535)`, before → after | 607.19 | 2.78 | −99.54 [−99.56, −99.52] | improvement |
| `rank(65534)` control | 614.71 | 648.20 | +5.45 [+0.13, +12.69] | inconclusive |
| Full-container range | 615.03 | 12.47 | −97.97 [−98.02, −97.92] | improvement |
| *Cache states, Docker/WSL2, `SingleShotTime`, ns per `rank(65535)` call (10 pairs)* | | | | |
| Precomputed cardinality | 964.04 | 8.74 | −99.09 [−99.43, −98.43] | improvement |
| First call, invalid (−1) | 857.01 | 714.60 | −16.62 [−38.14, +8.92] | inconclusive |
| Next call after priming | 660.44 | 8.42 | −98.73 [−98.99, −98.37] | improvement |

large reduction, while the near-boundary query was inconclusive (upper bound near zero and sensitive to the bootstrap seed): the query boundary matters more than the input's origin.

### 5.4 The first call on an invalid cache

The old revision scans on every call and leaves the cardinality at −1; the new one stores it during the first invalid-cache call and reads it afterwards. The lowest block of Table 3 and Fig. 2 show the consequence. With a precomputed cache and on the call after priming, the new revision was about 99% faster. On the first invalid call the estimate was −16.62%, but [−38.14, +8.92] crosses zero: *no first-call improvement is established* under this protocol. Inside the new revision, the next call took 98.82% less time than the first [−99.13, −98.43] (a comparison of states, not releases). Fork means vary widely (Fig. 2). In the old revision, whose code path is the same scan in all three states, the state-to-state differences ranged from −11.10% to −31.49% with all intervals crossing zero. We read this as the resolution limit of the single-shot design. Absolute values differ from the `AverageTime` blocks because the benchmark mode differs; they must not be merged.

### 5.5 Replication on native Windows

In the native block of Table 3, both large improvements match Docker/WSL2 in direction and size. The near-boundary control, however, changed by +5.45% [+0.13, +12.69]: the interval excludes zero, so the native measurement was slower in this run, but it crosses the 5% threshold, so a practically relevant regression is *not* established. OS, JDK build, scheduling and isolation changed together, so the difference cannot be attributed to Windows alone.

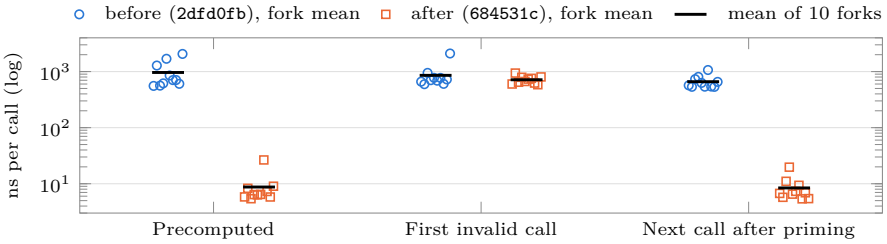



**Fig. 2.** Fork means of the cache-state study (10 forks per revision and state; raw data: `docker-lazy/forks.csv`). The new revision is fast only once the cardinality is valid; on the first invalid call both revisions overlap.

## 6 Discussion and Threats to Validity

*Answers to the research questions. RQ1:* Paired fresh-process rounds with an interval and a practical threshold separated large known effects from noise on two machines and avoided false claims such as the NumPy unchanged-version case, but only when the effect was large relative to the host's noise; unchanged code was inconclusive on the noisier host and small effects often were. *RQ2:* Contention changed time by about 50–100%; mitigations lowered mean time but not demonstrably variability; warmup and JIT settings changed results up to tenfold. *RQ3:* The RoaringBitmap optimization was reproduced in two environments, and control and ablation localize its mechanism; its benefit holds for valid cached cardinalities and repeated calls, not demonstrably for the first call on an invalid cache.

*Relation to the literature.* Our results confirm that independent repetition and uncertainty are necessary [1,2]; the inconclusive controls on the noisier host illustrate that required repetition depends on the variance structure [2], to which our fixed design did not adapt. The cache-state result extends Georges et al.'s distinction of execution phases: *logical* program state, not only VM warmup, defines what is measured. As Laaber and Leitner observe, a stable benchmark need not be relevant: the near-boundary query is stable but misses the optimization. As in Heger et al. and Coz, an intervention is stronger evidence than a timing difference or a profile.

*Threats to validity. Internal:* contention, cache locality, JIT and background activity on shared hosts are not fully controlled; forks may be dependent under drift. *Construct:* operation, process and JMH times differ; the single-shot design reuses arrays. *Statistical:* 8–20 pairs, marginal percentile intervals, no multiple-comparison correction, and synthetic coverage below nominal in some cells. *External:* one real Java case, a selected boundary query, a source-text input proxy, and replication of the real case on one physical machine only; no end-to-end latency, memory, energy or native compiler optimization was measured. The literature review is targeted, not systematic.

*Implications for software engineering.* A classification should start an investigation, combining a stated objective, correctness checks, a defined workload and state, environment records, diagnostics and a rule for inconclusive results.

## 7 Conclusion and Future Work

Starting from a preliminary NetworkX study and prototype, we extended a paired, uncertainty-aware regression detector into a workflow that measures, diagnoses and verifies performance changes in Python and Java. The answer to the seminar question is conditional: measurement is reliable when workload, program state and environment are specified, independent runs are compared with intervals and a practical threshold, and controls and interventions separate mechanism from artifacts. Under these conditions the RoaringBitmap optimization gave a reproducible, explained improvement for cached queries; its first-call effect remained uncertain. Future work: the planned analysis of regressions across NetworkX history, variance-adaptive repetition [2], replication on independent machines, more real optimizations, and detector calibration on known real regressions.

*Provenance and use of AI tools.* The preliminary report, presentation and prototype were produced by the seminar team [15,16]; their code, data and attribution are preserved unchanged [17]. [Individual author contributions to the prototype and to the extension: to be confirmed by the team before submission.] OpenAI Codex helped implement the extension, run experiments and tests, and write technical reports; OpenAI ChatGPT helped compare sources and prepare an earlier draft of this paper; Anthropic Claude helped integrate the preliminary report and new results, recompute reported estimates from archived raw files as a consistency check, and edit the manuscript [19]. The tools are neither authors nor independent validators; the authors are responsible for the content. *Artifact availability:* code and evidence [17] are not publicly hosted; the authors hold the archived snapshots. Saved results can be re-checked without Docker via `upgrade/java/final/study.py verify <run>`; a full re-run needs Docker and JDK 21 (`docs/REPRODUCIBILITY_UPGRADE.md`).

## References

1. Georges, A., Buytaert, D., Eeckhout, L.: Statistically rigorous Java performance evaluation. In: Proc. OOPSLA 2007, pp. 57–76. ACM (2007). https://doi.org/10.1145/1297027.1297033
2. Kalibera, T., Jones, R.: Rigorous benchmarking in reasonable time. In: Proc. ISMM 2013, pp. 63–74. ACM (2013). https://doi.org/10.1145/2464157.2464160
3. Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F.: Producing wrong data without doing anything obviously wrong! In: Proc. ASPLOS 2009, pp. 265–276. ACM (2009). https://doi.org/10.1145/1508244.1508275
4. Barrett, E., Bolz-Tereick, C.F., Killick, R., Mount, S., Tratt, L.: Virtual machine warmup blows hot and cold. Proc. ACM Program. Lang. **1**(OOPSLA), Article 52 (2017). https://doi.org/10.1145/3133876

5. Chen, J., Shang, W.: An exploratory study of performance regression introducing code changes. In: Proc. ICSME 2017, pp. 341–352. IEEE (2017). https://doi.org/10.1109/ICSME.2017.13
6. de Oliveira, A.B., Fischmeister, S., Diwan, A., Hauswirth, M., Sweeney, P.F.: Perphecy: Performance regression test selection made simple but effective. In: Proc. ICST 2017, pp. 103–113. IEEE (2017). https://doi.org/10.1109/ICST.2017.17
7. Chen, J., Shang, W., Shihab, E.: PerfJIT: Test-level just-in-time prediction for performance regression introducing commits. IEEE Trans. Softw. Eng. **48**(5), 1529–1544 (2022). https://doi.org/10.1109/TSE.2020.3023955
8. Laaber, C., Leitner, P.: An evaluation of open-source software microbenchmark suites for continuous performance assessment. In: Proc. MSR 2018, pp. 119–130. ACM (2018). https://doi.org/10.1145/3196398.3196407
9. dos Santos, L.B.R., de Souza, E.F., Endo, A.T., Trubiani, C., Pinciroli, R., Vijaykumar, N.L.: Performance regression testing initiatives: a systematic mapping. Inf. Softw. Technol. **179**, 107641 (2025)
10. NetworkX Developers: NetworkX ASV benchmarks. https://github.com/networkx/networkx/blob/main/benchmarks/README.md, last accessed 2026/10/19
11. Heger, C., Happe, J., Farahbod, R.: Automated root cause isolation of performance regressions during software development. In: Proc. ICPE 2013, pp. 27–38. ACM (2013). https://doi.org/10.1145/2479871.2479879
12. Curtsinger, C., Berger, E.D.: Coz: Finding code that counts with causal profiling. In: Proc. SOSP 2015, pp. 184–197. ACM (2015). https://doi.org/10.1145/2815400.2815409
13. RoaringBitmap project: Pull request #845, Optimize rank(0xFFFF) to use container cardinality. https://github.com/RoaringBitmap/RoaringBitmap/pull/845, last accessed 2026/11/05
14. Seminar Topics, Advanced Software Engineering, Topic 9, p. 12. Course material, HAW Kiel (2026)
15. Bashayev, A., Nguyen, T.T., Jung, D.: An empirical study of performance regressions in NetworkX. Preliminary seminar report, HAW Kiel (19 October 2026, unpublished)
16. Bashayev, A., Nguyen, T.T., Jung, D.: Software performance measurement and optimization. Seminar presentation (Team 3), HAW Kiel (2026); prototype: https://github.com/Bashshesh/benchmarking
17. PerfLens code and evidence archive (raw data, JMH JSON, receipts, handoff record `docs/HANDOFF.md`), unpublished ZIP snapshots: `PerfLens-review-20261105.zip`, SHA-256 `7612d2e17e3140b8ebd310a371d09eab7858e6c27d4ae023c12424feeb5ce965`; `PerfLens-final-review.zip`, SHA-256 `28af5db33f2f7fedb6a6747ed7b9c84fde743fcb7e2db1291f6906acf3804b00`. RoaringBitmap commits `2dfd0fb994468c8d416dea3d217dc40ca7d6d145` and `684531c13877fbdacccf02262b32e6faf889d76f`
18. Plan for whole lab: repository criteria C1–C10, change selection. Internal team document (`Plan for whole lab.docx`), HAW Kiel (2026)
19. OpenAI: ChatGPT, Codex; Anthropic: Claude. AI assistants, used October–November 2026